Distributed and Parallel Systems - Truck Platooning

Milestone 1: Tasks 1 and 2

Scope and assumptions

This is a first-version model for a classroom simulation of one lead truck and three followers. The lead truck broadcasts status; each follower uses its own forward-facing distance sensor and speed estimate. Units are SI. Assumed communication update period: 100 ms; warning after 200 ms without a valid update; fallback after 300 ms.

Task 1A - Required data, signals and events

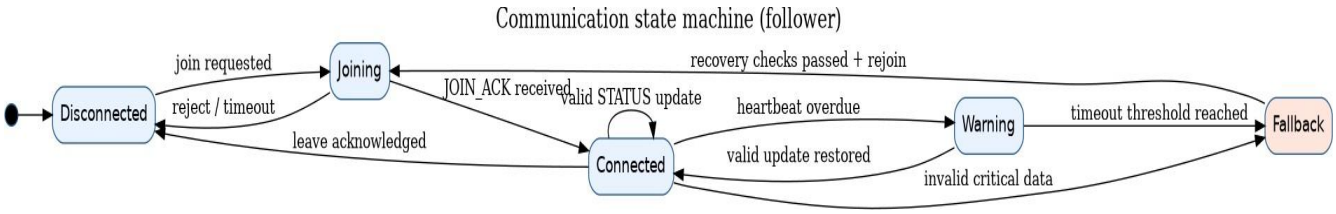
Periodic STATUS_UPDATE fields: protocol_version, sender_truck_id, platoon_id, sequence_number, timestamp_ms, speed_mps, acceleration_mps2, brake_active, emergency_flag, mode. Locally measured follower inputs: front_gap_m, relative_speed_mps, sensor_valid, own_speed_mps. Events: JOIN_REQUEST, JOIN_ACK, JOIN_REJECT, LEAVE_REQUEST, LEAVE_ACK, STATUS_UPDATE, EMERGENCY_BRAKE, HEARTBEAT_TIMEOUT, LINK_RESTORED, SENSOR_FAULT. The sender identity, sequence number and timestamp help receivers reject unexpected, duplicate and stale messages.

Task 1B - Proposed communication protocol

For the simulation, we use UDP datagrams for frequent vehicle status broadcasts because fresh data is more valuable than delayed retransmissions. Application-level JOIN_REQUEST/JOIN_ACK and LEAVE_REQUEST/LEAVE_ACK with timeout and bounded retries. Emergency-brake warnings are transmitted immediately and repeated briefly, but follower braking must never depend on message receipt. Validate payload lengths, units, sender identity, platoon membership, sequence numbers and freshness. Use a monotonic local timer for watchdogs. The network simulation should inject packet loss, delay, reordering and disconnections. Real vehicle-to-vehicle deployments may instead use ITS-G5 with GeoNetworking/BTP or C-V2X; UDP here is a project implementation choice.

Task 1C - Communication state machine

The follower starts Disconnected, sends a join request (Joining), and becomes Connected only after a valid acknowledgment. Valid STATUS_UPDATE messages refresh the watchdog. Missing updates enter Warning, and prolonged loss or invalid critical data enters Fallback. Returning to Connected requires validated recovery and rejoining; an ordinary packet arriving after timeout must not silently restore full platoon control.



Task 2A - Control behaviour and distance

Each follower computes a desired gap d_target = d0 + h*v_follower, where d0 is standstill clearance, h is time headway, and v_follower is current speed. Example ONLY: d0 = 5 m, h = 2 s, v = 20 m/s gives d_target = 45 m. Let e = measured_gap - d_target and relative_speed = v_preceding - v_follower. A conceptual controller uses a_cmd = clamp(kp*e + kd*relative_speed, -b_max, a_max), with acceleration/braking saturation, rate limits and hysteresis to avoid switching rapidly. Use the local distance sensor as the primary safety input; leader broadcasts improve

anticipation. The formula is an illustrative control law, not a mathematical guarantee of safe distance.

Task 2B - Communication failure, robustness and stability

If a heartbeat is overdue, mark remote data stale; on timeout, switch from cooperative following to sensor-only adaptive following, increase the target time headway, and cap acceleration. If the measured gap is critically short, use emergency braking within simulated limits. If the sensor is invalid as well, request driver intervention and transition to a controlled safe-stop strategy as feasible. Avoid automatically resuming platooning until the link and sensor checks pass. Stability cannot be claimed from a state machine alone: verify collision avoidance, bounded gap error and string stability (disturbances should not amplify from one follower to the next) through simulation. Test abrupt lead braking, packet losses, 0.5-2 s outages, jitter, sensor faults and recovery; log minimum gap, spacing error, speed, acceleration and mode transitions.

Task 2C - Control state machine

Standby -> Following after a validated join. Following -> IncreaseSpeed when the gap is above the permitted band, and -> ReduceSpeed when below. Both return to Following when the gap is restored. Any following mode -> SensorFallback on communication timeout. SensorFallback -> SafeStop on critical gap or invalid sensors; return to Following only after recovery validation.

